Chapter 24: Recursion

Learning objectives

By the end of this chapter you should be able to:

- show understanding of the essential features of recursion
- show understanding of how recursion is expressed in a programming language
- trace recursive algorithms
- write recursive algorithms
- show understanding of when the use of recursion is beneficial
- show awareness of what a compiler has to do to translate recursive programming code, including the use of stacks and unwinding.

24.01 Concept of recursion

In mathematical logic and computer science, a function or procedure is said to be a **recursive routine** if it is defined in terms of itself.

KEY TERM

Recursive routine: a function or procedure defined in terms of itself

The classic mathematical example is the factorial function, n!, which is defined in Figure 24.01. This definition holds for all positive whole numbers.

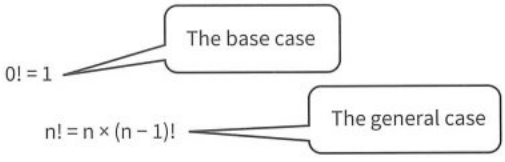


Figure 24.01 Mathematical definition of the factorial function

Figure 24.02 shows expressions of the factorial function for the first four numbers.

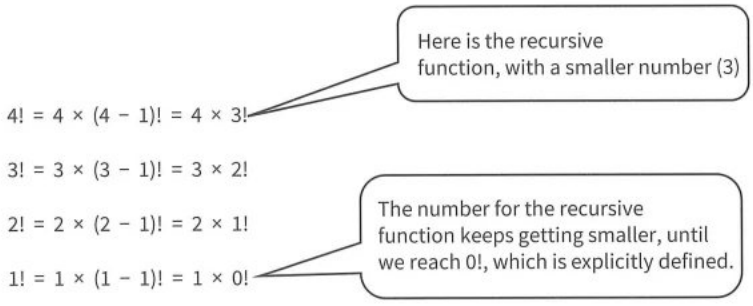


Figure 24.02 Expressions of the factorial function

Because 0! = 1:

$$4! = 4 \times 3 \times 2 \times 1 \times 1 = 24$$

Recursive solutions have a **base case** and a **general case**. The base case gives a result without involving the general case. The general case is defined in terms of the definition itself. It is very important that the general case must come closer to the base case with each recursion, for any starting point.

KEY TERMS

Base case: an explicit solution to a recursive function

General case: a definition of a recursive function in terms of itself

24.02 Programming a recursive subroutine

WORKED EXAMPLE 24.01

Coding the factorial function

We could program the function `Factorial` iteratively, using a loop:

```
FUNCTION Factorial(n : INTEGER) RETURNS INTEGER
    Result ← 1
    FOR i ← 1 TO n
        Result ← Result * i
    NEXT i
    RETURN Result
ENDFUNCTION
```

Alternatively, we can define the function `Factorial` recursively (Figure 24.03).

```
FUNCTION Factorial(n : INTEGER) RETURNS INTEGER
    IF n = 0
      THEN
        Result ← 1
      ELSE
        Result ← n * Factorial(n - 1)
    ENDIF
    RETURN Result
ENDFUNCTION
```

This is the base case

This is the general case

This is the recursive call

Figure 24.03 The factorial function defined recursively

The recursive pseudocode resembles the original mathematical definition of the factorial function. The dry run in Figure 24.05 (Section 24.03) shows how this works.

Discussion Point:

Carefully examine the two solutions to the factorial function. What happens if the iterative function is called with parameter 0? What happens if the recursive function is called with parameter 0? What changes would need to be made so the mathematical definition holds for all values of n?

When writing a recursive subroutine, there are three rules you must observe. A recursive subroutine must:

- have a base case
- have a general case
- reach the base case after a finite (limited) number of calls to itself.

TASK 24.01

Write program code to implement the recursive algorithm for the factorial function.

Question 24.01

What happens when the function is called with `Factorial(-2)`? Which rule is not satisfied?

WORKED EXAMPLE 24.02

Coding a recursive procedure

Consider a procedure to count down from a given integer. We can write the solution as an iterative algorithm:

```
PROCEDURE CountDownFrom(n : INTEGER)
    FOR i ← n DOWNTO 0
        OUTPUT i
    NEXT i
ENDPROCEDURE
```

We can also write the solution as a recursive algorithm. Consider what happens after the first value has been output. The remaining numbers follow the same pattern of counting down from the next smaller value. The base case is when n reaches 0. 0 will be output but no further numbers. The general case is outputting n and then counting down from (n – 1). This can be written using pseudocode:

```
PROCEDURE CountDownFrom(n : INTEGER)
    OUTPUT n
    IF n > 0
      THEN
        CALL CountDownFrom(n - 1)
    ENDIF
ENDPROCEDURE
```


24.03 Tracing a recursive subroutine

Tracing a recursive procedure

Dry-running the recursive procedure from Worked Example 24.02, we can complete a trace table as shown in Table 24.01.

Call number	Procedure call	OUTPUT	n > 0
1	CountDownFrom(3)	3	TRUE
2	CountDownFrom(2)	2	TRUE
3	CountDownFrom(1)	1	TRUE
4	CountDownFrom(0)	0	FALSE

Table 24.01 Trace table for `CALL CountDownFrom(3)`

It is more complex to trace a subroutine that contains statements to execute after the recursive call. Look at the slightly modified algorithm:

```
PROCEDURE CountUpTo(n : INTEGER)
    IF n > 0
      THEN
        CALL CountUpTo(n - 1)
    ENDIF
    OUTPUT n
ENDPROCEDURE
```

Note that the statements after `CALL CountUpTo(n - 1)` are not executed until control returns to this statement as the recursive calls unwind.

What is the effect of moving the `OUTPUT` statement to the end of the procedure? Figure 24.04 traces the execution of `CALL CountUpTo(3)`

Call number	Procedure call	n > 0	OUTPUT
1	CountUpTo(3)	TRUE	
2	CountUpTo(2)	TRUE	
3	CountUpTo(1)	TRUE	
4	CountUpTo(0)	FALSE	0
(3)	CountUpTo(1)	TRUE	1
(2)	CountUpTo(2)	TRUE	2
(1)	CountUpTo(3)	TRUE	3

Base case reached

Recursive calls unwind

Figure 24.04 Trace table for `CALL CountUpTo(3)`

When the base case is reached, the fourth call of the procedure is complete and the procedure is exited. Control then passes back to the third call and so on. Note how we show the trace as the recursive calls unwind. Don't go back up the table and fill in the OUTPUT column as this will not make it clear enough when the output occurred.

Tracing a recursive function

A recursive function has a statement after the recursive call to itself: the `RETURN` statement. Again we show what happens when the recursive calls unwind by filling in more rows in the trace table. Let's consider the factorial function again.


```
FUNCTION Factorial(n : INTEGER) RETURNS INTEGER
    IF n = 0
      THEN
        Result ← 1
      ELSE
        Result ← n * Factorial(n - 1)
    ENDIF
    RETURN Result
ENDFUNCTION
```

Call number	Procedure call	n = 0	Result	Return value
1	Factorial(4)	FALSE	4 * Factorial(3)	
2	Factorial(3)	FALSE	3 * Factorial(2)	
3	Factorial(2)	FALSE	2 * Factorial(1)	
4	Factorial(1)	FALSE	1 * Factorial(0)	
5	Factorial(0)	TRUE	1	1
(4)	Factorial(1)	FALSE	1 * 1	1
(3)	Factorial(2)	FALSE	2 * 1	2
(2)	Factorial(3)	FALSE	3 * 2	6
(1)	Factorial(4)	FALSE	4 * 6	24

Base case reached

Recursive calls unwind

Figure 24.05 Trace table for `CALL Factorial(4)`

Another way to illustrate how the function calls unwind is by framing each call with a box (see Figure 24.06). When the innermost box is completed the result is fed to the next one out. And so on until the outermost box has been completed.

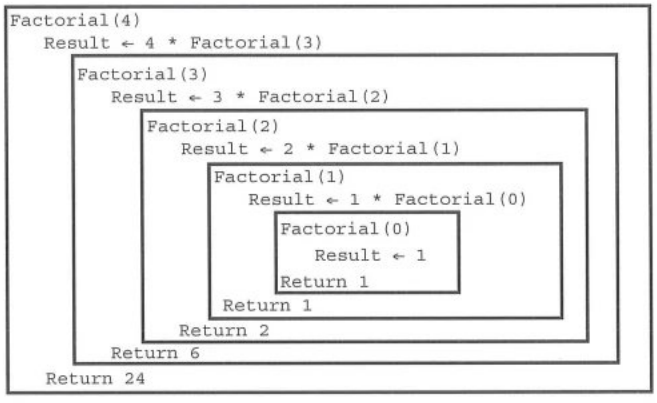


Figure 24.06 Diagrammatic view of recursive calls of `Factorial`

TASK 24.02

Consider the following recursive algorithm:

```
PROCEDURE X(n : INTEGER)
    IF (n = 0) OR (n = 1)
      THEN
        OUTPUT n
      ELSE
        CALL X(n DIV 2)
        OUTPUT(n MOD 2)
    ENDIF
ENDPROCEDURE
```

Dry-run the procedure call `X(19)` by completing a trace table. What is the purpose of this algorithm?

24.04 Running a recursive subroutine

Recursive subroutines can only be executed if the compiler produces object code that uses a stack to push return addresses and local variables when calling a subroutine repeatedly.

WORKED EXAMPLE 24.03

Running the factorial function

Consider the following program, written in pseudocode:

```
010  PROGRAM
020
030  FUNCTION Factorial(n : INTEGER) RETURNS INTEGER
040      IF n = 0
050        THEN
060          Result ← 1
070        ELSE
```

(Continued)

```
080           Result ← n * Factorial(n - 1)
090       ENDIF
100       RETURN Result
110   ENDFUNCTION
120
130   // main program
140
150   DECLARE Answer : INTEGER
160   Answer ← Factorial(3)
170   OUTPUT Answer
180
190   ENDPROGRAM
```

The first program statement to be executed is line 160. The actual parameter `n` has the value 3. The function call causes the return address to be put on the stack, as shown in Figure 24.07. Program execution jumps to line 30.

When line 80 is reached, the function call causes the return address to be stored on the stack, together with the current contents of the local variables. The locations used to store these values are referred to as a stack frame (represented by the blue borders in Figure 24.07). Each recursive call will add another stack frame to the stack until the base case is reached.

When the base case is reached, the result of the function call `Factorial(0)` is returned by pushing it onto the stack. The result is popped off the stack by the previous call of the function. With each return from a function call, the corresponding stack frame is taken off and the values of the local variables are restored. Eventually, control is returned to line 160 with the result of the function call on the top of the stack. The value of `Answer` is output in line 170.

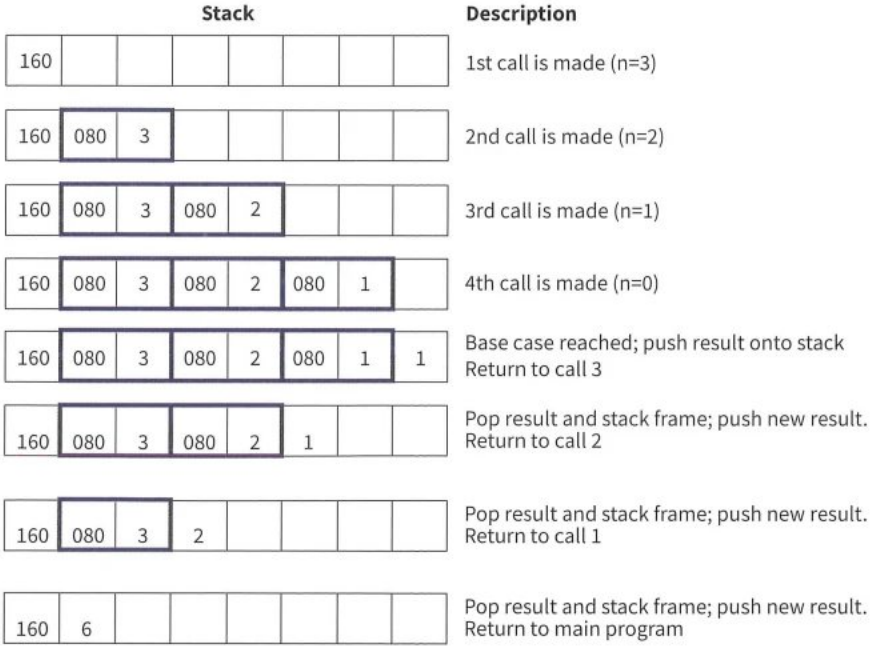


Figure 24.07 Stack contents during recursive calls of `Factorial`

TASK 24.03

Use your program code from Task 24.01 and add the main program as shown in Worked Example 24.03.

Amend your code in the following ways (line numbers are relative to the pseudocode in Worked Example 24.03):

- Add a global integer variable `CallNumber`
- Initialise `CallNumber` to zero (line 155).
- Increment `CallNumber` (line 35).
- Add a statement to output the values of `CallNumber` and `n` (line 36).
- Add a statement to output the value of `Result` (line 95).

Run the program and study the output.

24.05 Benefits and drawbacks of recursion

Recursion is an important technique in different programming paradigms (See Chapter 29, Section 29.08). When designing a solution to a mathematical problem that is recursive by nature, the easiest way to write a solution is to implement the recursive definition. Some optimising compilers will change a recursive subroutine to an iterative one when producing object code.

Recursive solutions are often more elegant and use less program code than iterative solutions. However, repeated recursive calls can carry large amounts of memory usage and processor time (see Section 24.04). For example, the procedure call `CountDownFrom(100)` will require 100 stack frames before it completes.

Reflection Point:

How can you tell from a function or procedure definition whether or not it is recursive?

Summary

- A recursive subroutine is defined in terms of itself.
- A recursive subroutine must have a base case and a general case.
- A recursive subroutine must reach the base case after a finite number of calls to itself.
- Each time a subroutine is called, a stack frame is pushed onto the stack.
- A stack frame consists of the return address and the values of the local variables.
- When a subroutine completes, the corresponding stack frame is popped off the stack.

Exam-style Questions

1 **a** Outline the differences between iteration and recursion. [2]

b Give **one** advantage and **one** disadvantage of using recursive subroutines. [2]

2 The following is a recursively defined function which calculates the result of $Base^{Exponent}$. For example, 2^3 is 8.

```
FUNCTION Power(Base: INTEGER, Exponent : INTEGER) RETURNS INTEGER
    IF Exponent = 0
      THEN
        Result ← 1
      ELSE
        Result ← Base * Power(Base, Exponent - 1)
    ENDIF
    RETURN Result
ENDFUNCTION
```

a What is meant by 'recursively defined'? [1]

b Trace the execution of the function call `Power(2,4)` showing for each re-entry into the `Power` function, the values passed to the function and the results returned. [6]

c Explain the role of the stack in the execution of the Power function. [3]

d Write a pseudocode non-recursive (iterative) version of the Power function. [6]

e **i** Give **one** reason why a non-recursive Power function may be preferred to a recursive one. [1]

ii Give **one** reason why a recursive Power function may be preferred to a non-recursive one. [1]

3 The following is a recursively defined function which calculates the *n*th integer in the sequence of Fibonacci numbers.

```
01  FUNCTION Fibonacci(n : INTEGER) RETURNS INTEGER
02      IF (n = 0) OR (n = 1)
03        THEN
04          Result ← 1
05        ELSE
06          Result ← Fibonacci(n - 1) + Fibonacci(n - 2)
07      ENDIF
08      RETURN Result
09  ENDFUNCTION
```

a **i** Which line is the base case? [1]

ii Which line is the general case? [1]

b Dry-run the function call `Fibonacci(4)`. [7]